46 WHICH QUERY TOOL TO USE — EACH CASE IN DETAIL, WITH EXAMPLES

theory/45 gave the short table:

Query type	Recommended
Basic CRUD	TypeORM repository
Filters, sorting, pagination	Repository or QueryBuilder
Joins and aggregations	QueryBuilder or raw SQL
Very complex analytics/reporting	Often raw SQL

This file takes each row one by one: what the query type MEANS, WHY that tool is recommended, a full EXAMPLE (NestJS service code), and the SQL TypeORM generates for it.

The three tools:

REPOSITORY	this.studentRepo.find(...)	methods + options objects
QUERYBUILDER	this.studentRepo.createQueryBuilder('s')...	chain of SQL-shaped methods
RAW SQL	this.dataSource.query('SELECT ...', [params])	you write the SQL string

Examples use this project's `students` table (student.entity.ts):

> id, name, age, email, course, createdAt, updatedAt plus, for joins, an imagined `courses` + `enrollments` setup (Section 3).

SECTION 1 — BASIC CRUD → TYPEORM REPOSITORY

WHAT IT MEANS:

```
Create, Read, Update, Delete ONE table, usually by id. The everyday
80% of an API: "add a student", "get student 3", "change age",
"delete student 3".
```

WHY REPOSITORY:

- one method per operation, no SQL strings at all
- returns typed Student objects (autocomplete, compile-time checks)
- save() handles createdAt/updatedAt, entity hooks, INSERT-vs-UPDATE
- parameters ($1) added automatically → SQL-injection safe
- QueryBuilder or raw SQL here would just be more code for the same SQL

EXAMPLE (this is exactly student-pg.service.ts)

```ts
// CREATE
async create(dto: CreateStudentDto) {
```

```
  const student = this.studentRepo.create(dto);   // object in memory
  return this.studentRepo.save(student);          // INSERT
}
SQL: INSERT INTO "students"("name","age","email","course","createdAt","updatedAt")
     VALUES ($1, $2, $3, $4, DEFAULT, DEFAULT) RETURNING "id","createdAt","updatedAt"

// READ ALL
findAll() { return this.studentRepo.find(); }
SQL: SELECT "id","name","age","email","course","createdAt","updatedAt" FROM "students"

// READ ONE
async findOne(id: number) {
  const s = await this.studentRepo.findOneBy({ id });
  if (!s) throw new NotFoundException(`Student ${id} not found`);
  return s;
}
SQL: SELECT ... FROM "students" WHERE "id" = $1 LIMIT 1

// UPDATE (load + merge + save -> returns the full updated row)
async update(id: number, dto: UpdateStudentDto) {
  const s = await this.studentRepo.preload({ id, ...dto });
  if (!s) throw new NotFoundException();
  return this.studentRepo.save(s);
}
SQL: SELECT ... WHERE "id" = $1;
     UPDATE "students" SET "age" = $1, "updatedAt" = CURRENT_TIMESTAMP WHERE "id" = $2

// UPDATE (quick, one statement, nothing returned)
await this.studentRepo.update(id, { age: 23 });
SQL: UPDATE "students" SET "age" = $1, "updatedAt" = CURRENT_TIMESTAMP WHERE "id" = $2

// DELETE
await this.studentRepo.delete(id);
SQL: DELETE FROM "students" WHERE "id" IN ($1)
```

Other one-table helpers:

findBy({ course: 'CS' })	WHERE course = $1	(many rows)
findOneByOrFail({ id })	throws if not found	
exists({ where: { email } })	true/false — e.g. "is this email taken?"	
count()	SELECT COUNT(*)	
increment({ id }, 'age', 1)	SET age = age + 1	
softDelete(id)	sets deletedAt instead of deleting (needs a @DeleteDateColumn)	

SECTION 2 — FILTERS, SORTING, PAGINATION → REPOSITORY OR QUERYBUILDER

WHAT IT MEANS:

```
The "list" screen: GET /students?course=CS&minAge=20&search=pri&sort=name&page=2
- FILTER     only rows matching conditions        (WHERE)
```

```
- SORT       in a chosen order                    (ORDER BY)
- PAGINATE   one page at a time + total count     (LIMIT / OFFSET)
```

WHY "REPOSITORY OR QUERYBUILDER":

Fixed, simple conditions	→ repository find options are enough
Conditions that depend on	→ QueryBuilder is cleaner: add each
which filters the user sent	.andWhere() only when needed

2a. REPOSITORY version (fixed filters)

```
  import { MoreThanOrEqual, ILike } from 'typeorm';

  findCsStudents(page = 1, limit = 10) {
    return this.studentRepo.findAndCount({
      where: {
        course: 'Computer Science',          // course = $1
        age: MoreThanOrEqual(20),            // age >= $2
        name: ILike('%pri%'),                // name ILIKE $3 (case-insensitive)
      },
      order: { name: 'ASC', id: 'DESC' },    // ORDER BY name ASC, id DESC
      skip: (page - 1) * limit,              // OFFSET
      take: limit,                           // LIMIT
    });
  }
  // returns [Student[], totalCount]

  SQL (two queries):
    SELECT ... FROM "students"
    WHERE "course" = $1 AND "age" >= $2 AND "name" ILIKE $3
    ORDER BY "name" ASC, "id" DESC LIMIT 10 OFFSET 0

    SELECT COUNT(DISTINCT("id")) FROM "students"
    WHERE "course" = $1 AND "age" >= $2 AND "name" ILIKE $3

OR conditions = array of where objects:
  where: [{ course: 'CS' }, { course: 'IT' }]     // course='CS' OR course='IT'
  (or: where: { course: In(['CS', 'IT']) })      // course IN ('CS','IT')
```

2b. QUERYBUILDER version (dynamic filters from the URL)

```
// controller: GET /student-pg/search?course=CS&minAge=20&search=pri&sort=age&order=DESC&page=2
search(q: { course?: string; minAge?: number; search?: string;
            sort?: 'name' | 'age' | 'createdAt'; order?: 'ASC' | 'DESC';
            page?: number; limit?: number }) {
  const page  = q.page ?? 1;
  const limit = q.limit ?? 10;

  const qb = this.studentRepo.createQueryBuilder('s');

  if (q.course) qb.andWhere('s.course = :course', { course: q.course });
  if (q.minAge) qb.andWhere('s.age >= :minAge', { minAge: q.minAge });
  if (q.search) qb.andWhere('(s.name ILIKE :search OR s.email ILIKE :search)',
                            { search: `%${q.search}%` });
```

```
  qb.orderBy(`s.${q.sort ?? 'createdAt'}`, q.order ?? 'DESC')
    .skip((page - 1) * limit)
    .take(limit);

  return qb.getManyAndCount();              // [rows, total]
}

SQL when only ?course=CS&search=pri was sent:
  SELECT ... FROM "students" "s"
  WHERE "s"."course" = $1 AND ("s"."name" ILIKE $2 OR "s"."email" ILIKE $2)
  ORDER BY "s"."createdAt" DESC LIMIT 10 OFFSET 0
```

Why QueryBuilder wins here: each filter is added only if present, and the OR inside brackets (name OR email) is easy to express.

SAFETY NOTE — sorting by a user-chosen column:

```
values (:course, :search) are parameters -> safe.
COLUMN NAMES can't be parameters, so `s.${q.sort}` must come from a
fixed list ('name' | 'age' | 'createdAt'), checked by the DTO with
@IsIn(['name','age','createdAt']). Never put raw user text there.
```

OFFSET vs CURSOR pagination:

OFFSET (skip/take)	"page 5"; simple, but slow on huge tables (Postgres still walks the skipped rows)
CURSOR / keyset	"rows after id 120": .where('s.id > :lastId', { lastId }).orderBy('s.id').take(10) fast at any depth — used for infinite scroll

SECTION 3 — JOINS AND AGGREGATIONS → QUERYBUILDER OR RAW SQL

WHAT IT MEANS:

```
JOIN         combine rows from two+ tables ("each student WITH their course")
AGGREGATION  calculate over many rows: COUNT, SUM, AVG, MIN, MAX,
             usually with GROUP BY ("how many students per course")
```

WHY QUERYBUILDER OR RAW SQL:

```
Repository can load relations, but can't GROUP BY, alias computed
columns, or filter on aggregates (HAVING). QueryBuilder can, while
staying TypeScript; raw SQL when it's easier to just write it.
```

Setup for the join examples (relations instead of a text `course`):

```
@Entity('courses')
export class Course {
  @PrimaryGeneratedColumn() id!: number;
  @Column() title!: string;
  @Column('int') fee!: number;
  @OneToMany(() => Student, (s) => s.course) students!: Student[];
}
```

// in Student:

```
@ManyToOne(() => Course, (c) => c.students, { nullable: true })
course?: Course;          // creates a "courseId" foreign key column
```

3a. JOIN with the REPOSITORY (simple: just load the relation)

```
this.studentRepo.find({ relations: { course: true } });

SQL: SELECT s.*, c.* FROM "students" s
     LEFT JOIN "courses" c ON c."id" = s."courseId"
Result: [{ id: 2, name: 'Ravi', course: { id: 1, title: 'CS', fee: 50000 } }, ...]
```

This is fine when you only need "the row + its related rows".

3b. JOIN with QUERYBUILDER (filter / sort on the joined table)

```
// students whose course fee > 40000, cheapest course first
this.studentRepo
  .createQueryBuilder('s')
  .innerJoinAndSelect('s.course', 'c')         // INNER JOIN + load c into s.course
  .where('c.fee > :fee', { fee: 40000 })
  .orderBy('c.fee', 'ASC')
  .getMany();

SQL: SELECT s.*, c.* FROM "students" "s"
     INNER JOIN "courses" "c" ON "c"."id" = "s"."courseId"
     WHERE "c"."fee" > $1 ORDER BY "c"."fee" ASC

leftJoinAndSelect   keep students WITHOUT a course (course = null)
innerJoinAndSelect  only students that HAVE a course
leftJoin (no Select) join only to filter, don't load the columns
```

3c. AGGREGATION with QUERYBUILDER

```
  // number of students and average age per course, only courses with 2+ students
  this.studentRepo
    .createQueryBuilder('s')
    .innerJoin('s.course', 'c')
    .select('c.title', 'course')
    .addSelect('COUNT(s.id)', 'students')
    .addSelect('ROUND(AVG(s.age), 1)', 'avgAge')
    .groupBy('c.title')
    .having('COUNT(s.id) >= :min', { min: 2 })
    .orderBy('students', 'DESC')
    .getRawMany();

  SQL: SELECT "c"."title" AS "course", COUNT(s.id) AS "students",
              ROUND(AVG(s.age), 1) AS "avgAge"
       FROM "students" "s" INNER JOIN "courses" "c" ON "c"."id" = "s"."courseId"
       GROUP BY "c"."title" HAVING COUNT(s.id) >= $1 ORDER BY "students" DESC

  Result: [{ course: 'CS', students: '12', avgAge: '20.5' }, ...]

  getRawMany() not getMany(): the result isn't a Student entity, it's
```

```
  computed columns. Note COUNT comes back as a STRING ('12') —
  Postgres bigint; convert with Number() if needed.

Same thing on the CURRENT table (course is plain text, no join):
  this.studentRepo.createQueryBuilder('s')
    .select('s.course', 'course').addSelect('COUNT(*)', 'total')
    .groupBy('s.course').getRawMany();
  -> [{ course: 'Computer Science', total: '1' }, { course: null, total: '1' }]
```

3d. Same aggregation as RAW SQL

```
  this.dataSource.query(`
    SELECT c.title AS course, COUNT(s.id) AS students, ROUND(AVG(s.age), 1) AS "avgAge"
    FROM students s
    JOIN courses c ON c.id = s."courseId"
    GROUP BY c.title
    HAVING COUNT(s.id) >= $1
    ORDER BY students DESC
  `, [2]);

QueryBuilder or raw here? Both fine. QueryBuilder if parts are
dynamic or you want to stay type-aware; raw SQL if you already wrote
and tested the query in Supabase's SQL editor.
```

SECTION 4 — VERY COMPLEX ANALYTICS / REPORTING → OFTEN RAW SQL

WHAT IT MEANS:

```
Dashboards and reports: rankings, running totals, month-by-month
trends, "top N per group", comparisons with previous periods. These
use advanced SQL:
  CTE (WITH ...)        name a sub-query and reuse it, step by step
  WINDOW FUNCTIONS      RANK(), ROW_NUMBER(), SUM() OVER (...) —
                        calculate across rows WITHOUT collapsing them
  date_trunc, FILTER, generate_series, JSONB operators ...
```

WHY RAW SQL:

QueryBuilder can technically express some of it, but becomes long and harder to read than the SQL itself. A 15-line SQL query you can test directly in Supabase → SQL Editor is clearer than 30 chained methods.

4a. Window function: rank students by age inside each course

```
this.dataSource.query(`
  SELECT name, course, age,
         RANK() OVER (PARTITION BY course ORDER BY age DESC) AS rank_in_course
  FROM students
`);
-> [{ name: 'Ravi', course: 'CS', age: 22, rank_in_course: '1' }, ...]
PARTITION BY = "restart the ranking for each course"
```

4b. CTE + window: youngest student per course (top-1 per group)

```
this.dataSource.query(`
  WITH ranked AS (
    SELECT s.*, ROW_NUMBER() OVER (PARTITION BY course ORDER BY age) AS rn
    FROM students s
  )
  SELECT id, name, course, age FROM ranked WHERE rn = 1
`);
```

4c. Monthly signup report with a running total

```
this.dataSource.query(`
  WITH monthly AS (
    SELECT date_trunc('month', "createdAt") AS month, COUNT(*) AS signups
    FROM students
    WHERE "createdAt" >= $1
    GROUP BY 1
  )
  SELECT to_char(month, 'YYYY-MM') AS month,
         signups,
         SUM(signups) OVER (ORDER BY month) AS running_total
  FROM monthly
  ORDER BY month
`, ['2026-01-01']);

-> [{ month: '2026-09', signups: '4', running_total: '4' },
    { month: '2026-10', signups: '2', running_total: '6' }]
```

Notes:

- "createdAt" needs double quotes in raw SQL (camelCase column, theory/44)
- still use $1 for any value from the user
- returns plain objects; numbers from COUNT/SUM come back as strings
- for heavy reports used often: create a Postgres VIEW (or materialized view) with this SQL, then map a read-only @ViewEntity to it, so the service code goes back to simple repository calls

SECTION 5 — HOW TO DECIDE (A QUICK FLOW)

```
One table, by id or simple fields?
    -> REPOSITORY  (find, findOneBy, save, update, delete)

A list with filters / sort / paging?
    fixed conditions          -> REPOSITORY find options
    conditions vary per request -> QUERYBUILDER (.andWhere only if sent)

Need data from another table?
    just load the related rows  -> REPOSITORY  relations: { course: true }
    filter/sort on the related table -> QUERYBUILDER join

COUNT / SUM / AVG / GROUP BY / HAVING?
    -> QUERYBUILDER + getRawMany()   (or raw SQL)
```

```
Window functions, CTEs, reports, Postgres-only features?
    -> RAW SQL via dataSource.query()  (test it in Supabase SQL Editor first)
```

Same request, three tools — "students older than 20, by name":

```
  Repository:   find({ where: { age: MoreThan(20) }, order: { name: 'ASC' } })
  QueryBuilder: createQueryBuilder('s').where('s.age > :a', { a: 20 }).orderBy('s.name').getMany()
  Raw SQL:      dataSource.query('SELECT * FROM students WHERE age > $1 ORDER BY name', [20])
All produce almost the same SQL. Pick the SIMPLEST one that can
express the query.
```

⚠ COMMON MISTAKES

- building SQL with `${userInput}` in raw queries or .where() strings → SQL injection. Always :param / $1.
- user-controlled sort column without a whitelist (@IsIn)
- getMany() on an aggregate query → empty/wrong objects; use getRawMany()
- forgetting COUNT/SUM return strings in Postgres
- loading all rows and filtering in JavaScript instead of WHERE in SQL
- N+1 queries: looping over students and querying each one's course → use a join / relations instead (1 query instead of N+1)
- unquoted camelCase columns in raw SQL ("createdAt")

★ BOTTOM LINE

Start with the Repository; move to QueryBuilder when the query must change per request or needs joins/aggregates; drop to raw SQL for real analytics (CTEs, window functions). Whatever the tool, values always go in parameters, and use logging: true to see the SQL each one produces.

See also: **theory/45-typeorm-vs-writing-sql.txt** (the short version), **theory/43-where-sql-queries-come-from.txt** (generated SQL, logging, transactions), **theory/37-sql-cheatsheet.txt** (SQL syntax), **theory/38-mongodb-relationships.txt** (the Mongo side of joins)